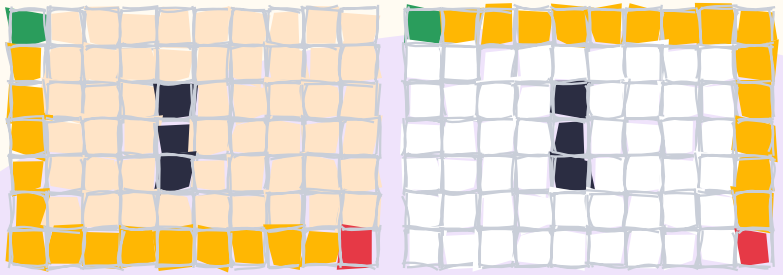
Tech Leaders · Class 21

Pathfinding Visualiser: BFS and A*

How do maps, games and robots find the shortest way?

Graphs | BFS | A* | Compare



BFS: 67 cells

A*: 16 cells

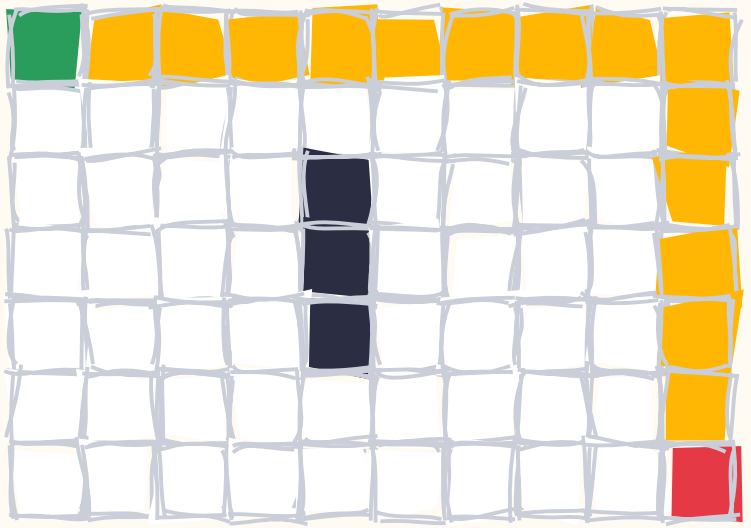
Did you know?
A* was invented in 1968 for Shakey, one of the first mobile robots. It's still inside game AI and route planners today.

Let's think...

How do map apps find routes so fast?

Millions of roads

Karachi has hundreds of thousands of road segments. Checking every possible route would take longer than the age of the universe. Yet your phone finds a good route almost instantly. What's the trick?



Check every route?

Guess?

Search smart!

Today: you build a **Pygame visualiser** that animates **BFS** and **A*** searching a maze, and measure which is smarter.

Learning objectives

By the end of this class I can...

1		**Grids are graphs:** model a map as a **graph**: cells are nodes, moves are edges	**Proof:** I can list a cell's neighbours in code
2		**BFS:** explain and code **breadth-first search** with a queue, visited set and came_from	**Proof:** my BFS finds the shortest path around walls
3		**A*:** code **A*** with a priority queue and **f = g + h** using Manhattan distance	**Proof:** my A* finds the same path but explores fewer cells
4		**Visualise + compare:** animate both in **Pygame** and compare **cells explored** and time	**Proof:** my results table shows A* beating BFS on 3 maps

LEARN

Objective 3 · A* · video

Video time!



A* Search and Heuristics Intuition in 2 minutes

Anish Krishnan · about 2 minutes · youtu.be/71CEj4gKDnE

Watch for...

1. What does the **heuristic** do in A*?
2. Why does A* explore **fewer** squares?
3. When would A* find a **wrong** path?

Scan to watch

After the video: Which apps or games do you think use pathfinding like this?

Objective 1 · Grids are graphs

A grid is a graph

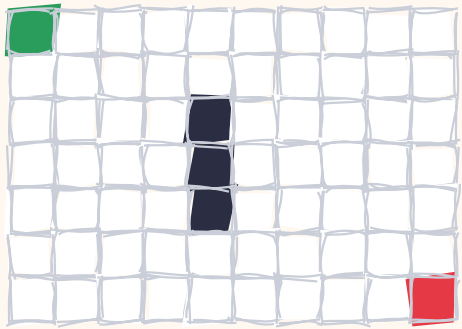
The idea

A **graph** = **nodes** + **edges**. In a grid map, each open cell (row, col) is a node; you can move up, down, left, right to open neighbours: those are edges.

Walls (#) simply have no edges.

neighbours(r, c) returns the open cells next to (r, c) that are inside the grid. Every pathfinding algorithm is built on this function.

Example: a metro map



A metro map ignores exact streets: stations are nodes, lines between them are edges. Pathfinding works on that simple picture.

Objective 2 · BFS

BFS: explore in rings

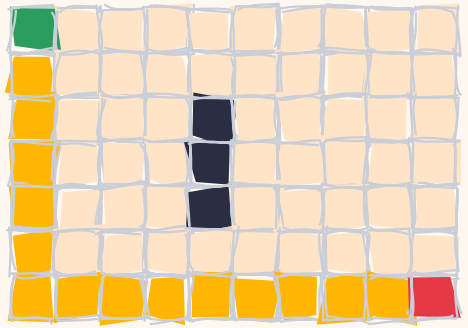
The idea

Breadth-First Search explores all cells 1 step away, then 2 steps, then 3... like ripples in a pond.

Use a **queue** (first in, first out). Keep a **visited** set so you never repeat, and **came_from** so you can walk the path back from the goal.

On a grid with equal step costs, BFS **guarantees the shortest path**. But it explores in **every** direction, even away from the goal.

Example: ripples in a pond



Drop a stone: the ripple reaches every point 1 cm away, then 2 cm... The first ripple to touch the goal took the shortest route.

Objective 3 · A*

A*: search towards the goal

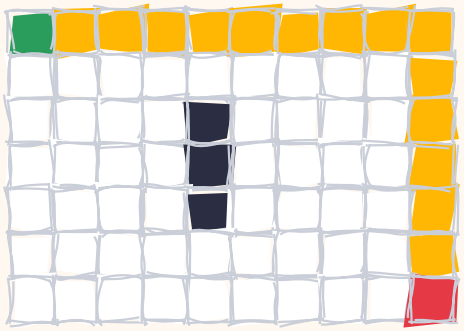
The idea

A* picks the cell with the lowest **f = g + h**:
g = real steps from the start so far
h = **heuristic** guess of steps to the goal. On a 4-direction grid: **Manhattan distance** |Δrow| + |Δcol|.

Use a **priority queue** (heapq) to always pop the smallest f.

If h **never overestimates** (admissible), A* still finds the **shortest** path, while exploring far fewer cells.

Example: walking towards a minaret you can see



In an unfamiliar town you head roughly towards the minaret (the heuristic), but you still count how far you've walked. That's g + h.

LEARN

Objective 4 · Visualise + compare

Measuring 'smarter'

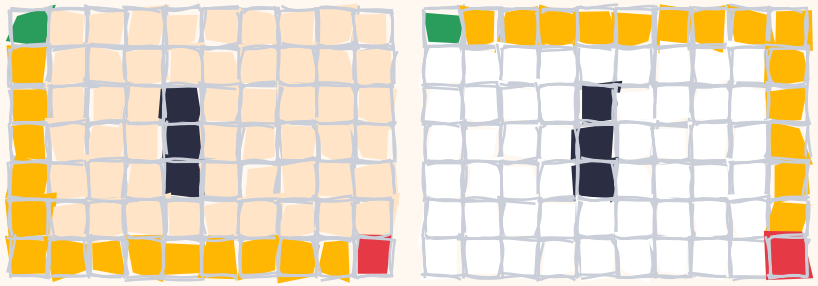
The idea

Both find a shortest path here. The difference is **work**: how many cells were **explored**, and how long it took.

On this maze: BFS explored **67** cells; A* explored **16**. Both paths have **15** steps.

Try open fields (A* wins big), dead-end mazes (closer), and a bad heuristic (h = 0 makes A* behave like Dijkstra/BFS; h × 5 is fast but may miss the shortest path).

Example: searching for keys



Searching every room equally (BFS) always works. Starting near where you last had them (A*) is faster, as long as your guess isn't misleading.

Real world

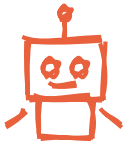
Pathfinding everywhere



Map apps
Route planners use A*-style search on road graphs



Game AI
Enemies chase you around walls with A*



Warehouse robots
Plan paths around shelves and each other



Networks
Internet routers find paths for your data

? A delivery rider in Lahore must visit 5 shops. Is the shortest path between two points enough? What else matters?

Weights
Real roads have different costs (traffic, distance). Then you need Dijkstra or A* with weighted edges.

LEARN

Quick quiz

Search sense

Question	Answer
1. BFS uses a stack or a queue?	
2. A* formula?	
3. Manhattan distance from (0,0) to (6,9)?	
4. Why store came_from?	
5. h overestimates. Risk?	

Answers

Cover! 1: queue · 2: f = g + h · 3: 15 · 4: to rebuild the path backwards · 5: may not find the shortest path

Part 2 · Make it

Now let's put it to work!

You know the ideas. Time to play, build and test. Use these slides while you work:

1. Human search
2. Our algorithm lab
3. Graph + BFS (as a generator)
4. A* with a priority queue
5. Animate and compare
6. Algorithm race
7. Algorithm designer challenges

Class game

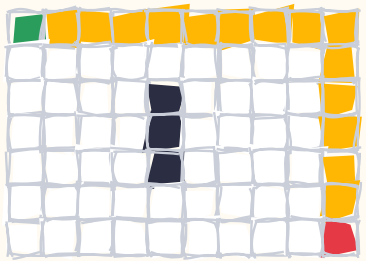
Human search

How to play

1. Masking-tape grid on the floor with chair 'walls'. One start square, one goal.
2. BFS team: everyone at distance 1 steps in, then distance 2… A scribe counts squares visited.
3. A* team: may only step onto the square with the smallest (steps so far + squares-to-goal) written on a sticky note.
4. Compare: who visited fewer squares? Did both find equally short paths?

Then discuss

When would the A* team's guess mislead them?



Big idea

A good guess saves work; a bad guess wastes it.

Build it

Our algorithm lab

Python + Pygame
Thonny or VS Code

collections.deque
Fast queue for BFS

heapq
Priority queue for A*

Map files
Mazes as text: # = wall, . = open

Counter
Cells explored + time.perf_counter()

Partner
Draw tricky mazes for each other

Design pattern: write the search as a generator (yield) so Pygame can draw each step as an animation.

Recursion limit
Don't write DFS recursively on big grids: Python stops at about 1000 calls deep. Use loops.

MAKE

Objective 2 · BFS · Build step 1 of 3

Graph + BFS (as a generator)

```
from collections import deque
GRID = [list(row) for row in open("maze.txt").read().split()]
R, C = len(GRID), len(GRID[0])

def neighbours(cell):
    r, c = cell
    for dr, dc in ((1, 0), (-1, 0), (0, 1), (0, -1)):
        nr, nc = r + dr, c + dc
        if 0 <= nr < R and 0 <= nc < C and GRID[nr][nc] != "#":
            yield (nr, nc)

def bfs(start, goal):
    queue = deque([start])
    came_from = {start: None}                # also our 'visited' set
    while queue:
        cur = queue.popleft()                # FIFO
        yield "explore", cur                 # for the animation
        if cur == goal: break
        for n in neighbours(cur):
            if n not in came_from:
                came_from[n] = cur
                queue.append(n)
    yield "path", rebuild(came_from, goal)

def rebuild(came_from, goal):
    path, n = [], goal
    while n is not None:
        path.append(n); n = came_from.get(n)
    return path[::-1]
```

yield

Each yield pauses the search so Pygame can draw one step. Then it continues.

came_from

Doubles as 'visited' AND lets us walk the path back.

maze.txt

Rows of . and #. Draw your own!

MAKE

Objective 3 · A* · Build step 2 of 3

A* with a priority queue

```python
import heapq

def manhattan(a, b):
    return abs(a[0] - b[0]) + abs(a[1] - b[1])

def astar(start, goal, h=manhattan):
    pq = [(h(start, goal), 0, start)]          # (f, -g, cell)
    g = {start: 0}
    came_from = {start: None}
    done = set()
    while pq:
        f, neg_g, cur = heapq.heappop(pq)      # smallest f first
        gc = -neg_g                            # ties: deepest g first
        if cur in done: continue
        done.add(cur)
        yield "explore", cur
        if cur == goal: break
        for n in neighbours(cur):
            new_g = gc + 1
            if new_g < g.get(n, float("inf")):
                g[n] = new_g
                came_from[n] = cur
                heapq.heappush(pq, (new_g + h(n, goal), -new_g, n))
    yield "path", rebuild(came_from, goal)
```

heapq

Always pops the cell with the smallest f = g + h.

Better route

If we find a cheaper g to a cell, we update it and push again.

Swap h

Try h = lambda a, b: 0. What does A* turn into?

MAKE

Objective 4 · Build step 3 of 3

Animate and compare

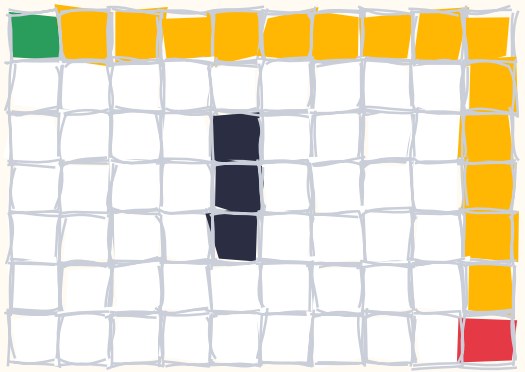
a Pygame loop: each frame, call **next(search)** once; colour explored cells orange, the path yellow.

b Keys: **B** = run BFS, **A** = run A*, mouse click = toggle a wall, **R** = reset.

c Count explored cells and time each search with **time.perf_counter()** (run without animation for timing).

d Test 3 maps: open field, maze, map with a long wall between start and goal.

e Bonus: allow 8-direction moves with cost √2 for diagonals (heuristic: octile distance).



Check: A results table: explored cells and time for BFS and A* on 3 maps.

Test like an engineer

Algorithm race

Map	Result	Notes
Open field: BFS / A* explored		
Maze: BFS / A* explored		
Long wall: BFS / A* explored		
Path lengths equal?		
A* with h = 0: explored		

Debug checklist

Path looks wrong? Check rebuild() walks from goal back to start, then reverse.
Infinite loop? You forgot to mark visited / done.
A* slower than BFS? On tiny maps the heap overhead can win: try bigger maps.

Fair comparisons use the SAME maps, start and goal for every algorithm.

Level up!

Algorithm designer challenges

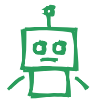
Choose your challenge



Weighted cells: mud costs 5, road costs 1. Use Dijkstra / A* with weights.



DFS: add depth-first search with a stack. Is its path shortest?



Robot link: send the path as F/L/R commands to a robot car.



Real map: use OpenStreetMap data (osmnx) to route between two places in your city.

Think like an engineer

Why is BFS guaranteed to find the shortest path on this grid, but not on a road map with different lengths?

Boss level

Make enemies in a Pygame game chase the player with A* every second.

Objective check

Can I really do it? Prove it.

01	What are the nodes and edges in your grid?	○ ○ ○
02	Explain BFS using the queue. Why does it find the shortest path here?	○ ○ ○
03	Explain f = g + h with a real cell from your run.	○ ○ ○
04	Show your comparison table. Which algorithm explored less, and on which map was the gap biggest?	○ ○ ○

Rate yourself

Colour one circle each. Red: not yet. Amber: with help. Green: I can explain it to a friend.

Teacher: Maze battle: swap maze files with another pair. Predict which algorithm wins, then run it.

Show what you know

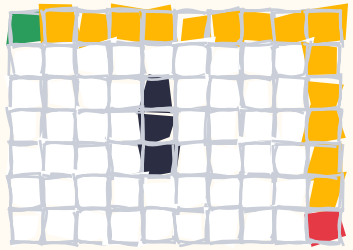
What did we learn?

- [x] Maps become graphs: nodes + edges
- [x] BFS: queue, explores in rings, shortest on equal costs
- [x] A*: priority queue on f = g + h
- [x] Admissible heuristics keep A* optimal
- [x] Measure work: cells explored and time

Exit ticket: what does A* become if h always returns 0?

At home

Open a map app and plan a route with a stop. How many choices did it compare?



Next class: Git & GitHub for Engineering Teams!